

Lab 7 Test Suite

I expect you to work independently on this assignment as if it was a take-home exam. Do not share your work, do not give help to others, nor receive help from others. I want this assignment to be your own work.

I am not supplying a test suite for Lab 7. As part of your lab grade, you are required to write your own test suite. When I grade the test suites, I will look for two things:

1. Logical progression of features - does each test add a small number of features to what has already been tested?
2. Coverage of the language - have you covered the features of the language such that any compiler that passes your test suite will likely also pass mine, or are there features that you have not tested? Or to put it better, are there bugs you have not looked for?

When designing tests, you need to put on your evil nemesis hat. The goal of the test suite is to find bugs, not to confirm that your compiler works. Don't think, "how can I show this feature works?", instead think, "how can I discover if this feature is broken?"

Your test suite must be delivered as a tar file attached to Canvas. The file needs to contain all your `.lang` and `.correct` files. Each `.lang` file must contain a comment at the top that lists your name and the purpose of the test. The names of your test files should imply the logical order (e.g. `test01`, `test02`, ..., but not `test_func`, `test_array`, ...). If you use a two-digit value for the number (with a leading zero for one-digit values), you will make my life slightly easier because then your tests will sort into proper order.

The syntax of the language allows some features that you are not required to test. The following list enumerates these:

1. You do not need to test floating point operations because `stackl` is an integer only machine.
2. You do not need to test long integers, because `stackl` is a 32 bit only machine.
3. Functions are not allowed to refer to variables declared in outer scopes.
4. You do not need to implement code for structs.
5. You do not need to test multi-dimension arrays (arrays of arrays).
6. You do not need to test the ability for a function to return anything other than an `int`.
7. You do not need to test the ability to pass an array to a function, but I will supply extra credit if you are able to do this.
8. The memory in `stackl` is limited. You do not need to test large memory data structures. If your compiler works on small ones, it should also work on large ones (assuming memory was available)

All features of the language not explicitly mentioned above should be tested.

The supplied `print` function will print a number followed by a newline. When you generate your `.correct` files, each number will be on a line by itself.